Detailed Individual Roles — 6 People, 36 Hours

Mesh algorithm is already built, so Person 2 and Person 6's mesh workload is lighter than before — that time gets redirected into the app's other features and into helping teammates. Every hour range below is a target, not a hard rule — if you finish early, move to the next task or check the "float to help" note.

Person 1 — Mobile: Login, Home, My Reports

In one sentence: You build every mobile screen except the report form and the mesh tab.

Hours	What you do	What "done" looks like
0–1	Team kickoff. Agree on the data shapes with everyone. Create your own git branch, push an empty first commit so you're on the board from hour 0.	Branch exists, contract agreed
1–3	Set up the Flutter app shell: bottom navigation bar with four tabs (Home, Mesh, My Reports, Profile), and the colour/font theme matching the dashboard mockups (the severity colours — red/orange/yellow/green — need to look the same here as they do on the web dashboards).	Tapping between tabs works, colours match the mockups
3–6	Build the **Login screen**: phone number entry, then a code entry screen (this can be fake for now — any 6-digit code works, no real SMS needed), then it drops you into Home.	You can "log in" with a fake code and land on Home
6–10	Build the **Home screen**: a status button the responder can tap to change (Available / En Route / Engaged / Resting / Off Duty), a coloured badge showing the area's current state (Alert / Emergency / Recovery), and a clock showing how long the emergency has been running. Use fake data for now.	Home screen looks and behaves like the mockup, with fake data
10–16	Build the **My Reports screen**: a scrollable list of everything this responder has submitted, each with a coloured status tag (Reported, Assigned, In Progress, Pending Verification, Closed, Reopened). Tapping one opens a detail view. Still fake data.	List renders, tapping an item shows detail
16–20	Swap fake data for real data — Person 5 should have the database ready by now. Connect Home and My Reports to it.	Both screens show real data pulled from the database
20–24	Handle the boring-but-important stuff: what the screen looks like while data is loading, what happens if the network call fails, empty states (e.g. "no reports yet").	App doesn't crash or look broken on slow/failed loads
24–28	Join Person 2 on the **Create Request** screen — it's the biggest, most important screen in the whole app, so two people finish it faster and it's worth pausing your own list for this.	Create Request screen functionally complete
28–32	Full run-through of the mobile app on a real phone — click through every screen, note anything broken.	A clean click-through with no crashes
32–35	Set up one "perfect" demo flow — realistic seed data so the app never looks empty when judges are watching.	Demo-ready phone, rehearsed once

Hours	What you do	What "done" looks like
35–36	Buffer / help wherever needed	

Commit habit: push after each screen reaches a working state, not just at the end of the day — aim for at least one commit every 2 hours.

Person 2 — Mobile: Mesh integration (using the ready algorithm) + Create Request

In one sentence: You plug the already-built mesh algorithm into the real phone app, then help build the report form.

Hours	What you do	What "done" looks like
0–1	Kickoff. Also — get the finished mesh algorithm code in front of you and actually read through it with Person 6 so you both understand how to call it.	You know exactly what functions/logic you're plugging in
1–2	Install the `flutter_p2p_connection` plugin and get through the Android setup — it needs several permissions in the manifest (location, nearby devices, storage). This step is fiddly, budget real time for it rather than rushing.	App builds and runs with the plugin installed, permissions granted on a real phone
2–4	Build the basic **Mesh tab** screen: a "start hosting" / "join as client" choice, and a list of nearby devices once connected.	Two phones can find and connect to each other
4–8	Wire the ready algorithm's logic underneath this screen — when a message needs to go out, it should use the existing routing/priority logic instead of you writing new logic from scratch.	A message typed on one phone shows up on the other, using the real algorithm
8–12	Test heavily with 2 real phones. The algorithm was likely only tested in a simulation before, so expect some bugs the first time it meets real hardware — permission timing issues, connection drops, etc. Fix as they come up.	Message send/receive is reliable across many repeated tests, not just once
12–16	If time and the algorithm support it, extend to a third phone and test whether messages can hop through one phone to reach another that's out of direct range.	3-phone relay demonstrated at least once — or, if it's not reliable, note that clearly and move on rather than losing hours here
16–18	**Mesh checkpoint.** Confirm with Person 6 that what you have is solid enough to demo. Because the algorithm was pre-built, this should land earlier than it would have from scratch.	A repeatable mesh demo exists
18–24	Move to the **Create Request screen** with Person 1: a grid of report-type icons (flooding, collapse, fire, etc.), a severity picker (four levels, one tap), a camera button that only opens the in-app camera (no photo-library option — this is deliberate), and it should automatically attach the phone's location and the time.	Screen is fully built, camera captures a photo with location+time attached
24–28	Connect the Create Request submit button to the database, including uploading the photo.	Submitting a report actually creates a real entry other screens can see

Hours	What you do	What "done" looks like
28–32	Stress-test the mesh demo again — more distance between phones, more time, different rooms — and fix anything that broke.	Confidence the demo will survive being shown live
32–35	Rehearse the exact steps you'll do on stage for the mesh part of the demo, in order, so there's no fumbling.	A scripted, practiced sequence
35–36	Buffer	

Commit habit: commit your mesh integration work separately from Person 1's screens, even when you're both touching the same app — keep your changes on your own branch/commits so it's clear what you built.

Person 3 — Web: Command Centre dashboard

In one sentence: You build the big-picture map screen that state-level controllers would use to see everything at once.

Hours	What you do	What "done" looks like
0–1	Kickoff	
1–3	Set up the React project and build the map component (this gets reused by Person 4's screen too, so build it as a shareable piece, not something baked only into your page).	A blank map renders on screen
3–6	Build the header (project name + account icon) and the status strip below it — a live clock, and a counter showing how many districts are currently in an emergency.	Header and status strip match the mockup
6–10	Add district shading to the map — each district coloured by its current state (red for emergency, orange/yellow for alert, otherwise neutral) — and incident points on top, coloured by how serious each one is.	Map shows districts and incidents with the right colours, using fake data
10–14	Build the side list of reported issues — filterable by type, severity, and status — and make selecting an item on the list highlight it on the map, and vice versa.	List and map stay in sync when you click either one
14–18	Add the two per-district buttons: "Contact CPOC" (opens a small panel with a name and phone number) and "View Uploaded Photos" (opens a gallery of geotagged photos from that district).	Both buttons open something real, even with sample data
18–22	Connect everything to the real database, so the map updates live as new incidents come in instead of needing a page refresh.	New test incidents appear on the map without reloading the page
22–26	Polish — fix any layout issues, make sure it looks right at the screen size you'll actually demo on, seed 2–3 realistic districts (one in full emergency, one in alert, others calm).	Dashboard looks demo-ready
26–30	Float over to Person 1 — help finish or polish the mobile app if they're behind.	
30–34	Join full-team integration testing — click through the whole system end to end with everyone else.	
34–36	Buffer / rehearsal	

Commit habit: commit each finished piece (header, map, list, buttons) as its own commit rather than one big dump at the end.

Person 4 — Web: CPOC dashboard + verification queue

In one sentence: You build the screen where a local coordinator checks incoming photos and approves or rejects them — this is what actually closes the loop in the demo.

Hours	What you do	What "done" looks like
0–1	Kickoff	
1–3	Set up your page using the shared map component Person 3 is building.	
3–6	Build the header and status strip — but scoped to one single area, not the whole state: this area's current status, the emergency clock, how many agency units are currently active here.	Matches the CPOC mockup
6–10	Build the map, but zoomed into just this one area (not the whole map like Person 3's screen) — showing agency positions and incident points, with pending-verification incidents visually flagged (e.g. a pulsing marker) so they stand out.	Map is area-scoped and pending items are visually distinct
10–14	Build the agencies list panel — which units are active, their status, synced with the map same as Person 3's incident list.	
14–19	Build the **verification queue** — this is the most important part of your screen. Each pending incident shows the original report photo next to the "resolved" photo side by side, with Approve and Reject buttons. If the resolved photo's location doesn't match the original incident's location closely enough, flag it visibly.	You can approve or reject a test incident and see it change state
19–22	Build the "linked reports" panel — when multiple people report the same thing, they show up grouped together under one entry instead of as separate items.	Grouped test data displays correctly
22–26	Connect Approve/Reject to the real database — approving should actually mark the incident closed and turn its dot green on both your map and Person 3's.	Full loop works: approve here → dot turns green everywhere
26–30	Float over to help Person 2 and Person 6 stress-test the mesh demo on real phones — extra hands are genuinely useful here.	
30–34	Full-team integration testing	
34–36	Buffer / rehearsal	

Commit habit: the verification queue is the single most demo-critical piece you own — commit it in small working steps so if something breaks late, you can roll back to the last good version instead of losing the whole feature.

Person 5 — Backend: Database setup

In one sentence: You build the database everyone else's screens pull from, using Supabase so you're not building login/storage/live-updates by hand.

Hours	What you do	What "done" looks like
0–1	Kickoff. Drive the conversation on exactly what data each screen needs — you're the one who has to build it, so make sure everyone agrees before you start.	Data shapes written down and agreed by all 6
1–3	Set up Supabase (either their free hosted option, which needs no server setup, or self-hosted if you'd rather run it locally).	A working Supabase project everyone can reach
3–6	Build the database tables: areas/districts, users, incidents, and photos, with the right fields on each (severity, status, location, timestamps, etc.)	Tables exist and match what the frontend teams expect
6–8	Set up login — simplest version: a fake OTP flow is fine for a hackathon, doesn't need to send real SMS.	Someone can "log in" and get an identity the app recognises
8–10	Write a script that fills the database with realistic fake data — several districts, a spread of incidents at different severities and statuses, a few users — so nothing ever looks empty during testing or the demo.	Running the script populates a full, realistic-looking dataset
10–12	Write the one clever piece of logic: when a new incident comes in, check if there's already a very similar one reported nearby and recently — if so, link them together automatically instead of showing two separate dots on the map.	Two similar test reports link into one
12–14	Write the approve/reject logic that Person 4's screen calls — approving closes the incident, rejecting reopens it with a reason attached. Also check that a "closing" photo's location roughly matches the original report's location, and flag it if not.	Person 4 can call this and see the right result
14–16	Make sure changes to the database show up live on the dashboards automatically, without anyone needing to refresh the page.	A change made in one place appears instantly somewhere else
16–20	Float to Person 3 and Person 4 — help them connect their screens to the real data, since you know the database best.	
20–24	Float to Person 1 and Person 2 — same, but for the mobile app.	
24–28	Fix anything in the database that integration testing turns up — expect some fields to need small changes as real screens start using them.	

Hours	What you do	What "done" looks like
28–32	Make sure the whole thing is reachable from wherever you're demoing from — either it's already hosted (if you used Supabase's free hosted option) or deployed somewhere reachable.	Everyone can hit the database from the demo location
32–36	Support final testing, buffer	

Commit habit: commit your database structure changes (migrations) as separate, named commits — "add incidents table," "add dedupe function" — so it's easy to see what was built when.

Person 6 — Backend: Mesh algorithm integration + floating support

In one sentence: You make sure the already-built mesh algorithm is properly wired in and demoable, then spend the rest of your time helping whichever team needs it most.

Hours	What you do	What "done" looks like
0–1	Kickoff. Sit down with Person 2 and go through the finished mesh algorithm together so you're both clear on how it works and how it should be used.	Shared understanding, no surprises later
1–3	If the algorithm exists as a standalone demo (for example, a browser-based visual showing the network reroute itself), get it fully working and polished — this is what you'll show judges to explain the idea clearly, separately from the real phones.	The standalone demo runs smoothly, controls work (e.g. a button to take a "node" offline and watch it reroute)
3–6	Continue polishing that standalone piece — smooth animations, clear labelling, no glitches — since this is a visual judges will actually watch closely.	It looks finished, not like a rough prototype
6–10	Pair with Person 2 to help get the algorithm's actual logic working inside the real phone app — if it was originally built for a browser demo, this might mean rewriting the core logic in the app's language, or figuring out the cleanest way to reuse it.	Person 2's real-device demo is using the same logic as the standalone version, not separate reinvented logic
10–14	Help Person 2 test the real 2-phone (and possibly 3-phone) demo — more hands catching bugs faster.	
14–16	**Mesh checkpoint** with Person 2 — confirm both the real-device demo and the standalone visual demo are solid.	Both pieces work reliably
16–20	Float to Person 5 — help with database logic, since backend work benefits from a second person once mesh is stable.	
20–24	Float to Person 3 and Person 4 — help wire up their dashboards if they're behind.	
24–28	Float wherever the team is furthest behind — check in with everyone and go help the weakest link, not just whoever asks first.	
28–32	Come back to mesh for final polish — make sure both demo pieces (real phones + standalone visual) still work after a full day of everyone else's changes.	
32–36	Support final rehearsal, buffer	

Commit habit: even though a lot of your time is "floating help," still commit your own changes under your own name rather than letting whoever you're helping commit for you.

Everyone: three checkpoints to actually stop and check

Hour 16: the core loop (report → approve → map turns red → resolve → verify → map turns green) should work at least once, even ugly. If it doesn't, that's the only priority until it does.

Hour 26: look at GitHub → Insights → Contributors as a team. If anyone's bar looks thin, fix it now, not at hour 35.

Hour 30: mesh demo (both the standalone visual and the real phones) needs to be reliably repeatable. After this hour, no new mesh features — testing and rehearsal only.